

Listeners asíncronos

El warm-up de caché — parte 2 de 2

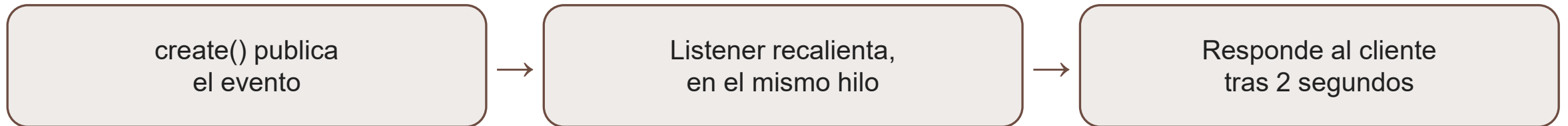
Qué pasaría si el listener no fuera asíncrono

El listener que vas a construir tiene que volver a llamar a `getTopNovedades()`, la misma consulta que tarda 2 segundos. Si Spring lo ejecutara de la forma más simple, lo haría en el mismo hilo que ha publicado el evento: el hilo de la petición HTTP.

El cliente, que solo quería su 201 Created, se quedaría esperando esos 2 segundos igualmente — el mismo coste que la caché pretendía evitar, solo que trasladado de sitio, no eliminado.

@Async rompe la cadena

❌ Listener normal, sin @Async



✅ Listener con @Async



@Async: qué hace exactamente

@Async le dice a Spring: no ejecutes este método en el hilo de quien lo llama — despáchalo a un pool de hilos aparte, y devuelve el control inmediatamente.

	Sin @Async	Con @Async
¿En qué hilo se ejecuta?	En el mismo hilo de quien lo llama	En un hilo del pool, aparte
¿Cuándo recupera el control quien llamó?	Cuando el método termina de ejecutarse	Inmediatamente, sin esperar a que termine

Por debajo, Spring envía la llamada a un `ExecutorService` (la escala de abstracción del primer apartado), y el método original se ejecuta en uno de esos hilos del pool.

La trampa clásica: @Async desde la misma clase

Atención

Spring solo puede interceptar una llamada que venga de fuera de la clase. Si un método llama directamente a otro método @Async de esa misma clase (`this.metodoAsync(...)`), no hay ninguna llamada externa que interceptar — se ejecuta como una llamada normal de Java, en el mismo hilo, y @Async se ignora EN SILENCIO, sin ningún error ni aviso.

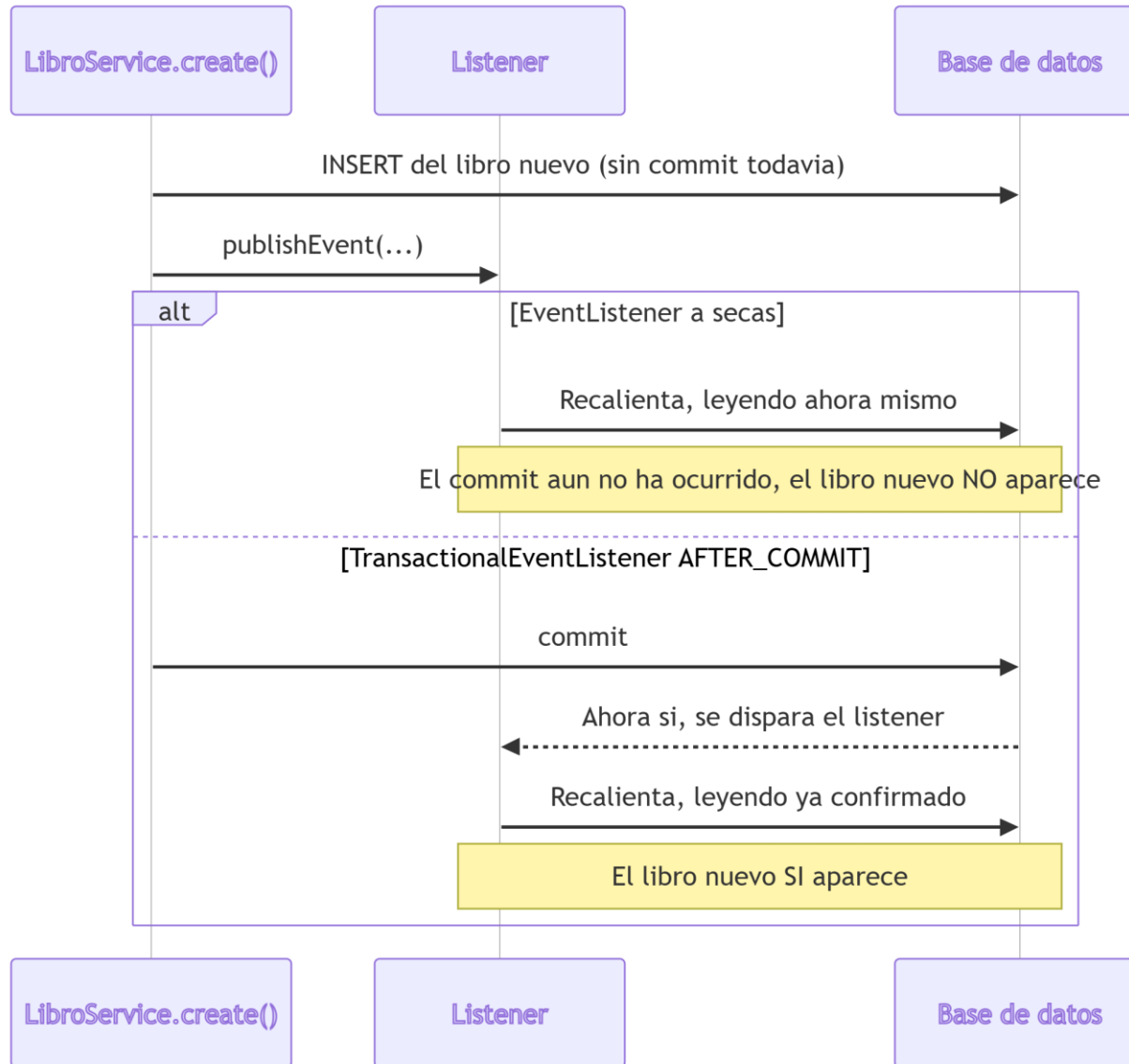
En este warm-up no va a pasar, porque el listener es un bean aparte al que Spring llama desde fuera — pero conviene tenerlo en la cabeza para el futuro.

Un problema sutil: el commit todavía no ha ocurrido

Si el listener se dispara antes de que la transacción de `LibroService.create()` haga commit, el hilo del warm-up podría leer la base de datos en un estado que todavía no incluye el libro recién creado — y recalentaría la caché con datos viejos.

`@TransactionalEventListener(phase = AFTER_COMMIT)` resuelve esto: espera a que la transacción que publicó el evento haga commit con éxito, y solo entonces ejecuta el listener.

Sin AFTER_COMMIT vs. con AFTER_COMMIT



La técnica de sincronización concreta

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Async
public void onLibrosBaratosInvalidado(
    LibrosBaratosInvalidadoEvent event) {
    // se ejecuta DESPUÉS del commit, EN OTRO HILO
}
```

Ya conoces `@Transactional` de Acceso a Datos — esto es la contrapartida del lado "quien reacciona a que una transacción ha terminado", no del lado "quien abre la transacción".

Estados de un hilo, ahora observables de verdad

Retoma el ciclo de estados del primer apartado, esta vez con el retraso artificial que simula el coste de recalcular los libros más baratos:

Momento	Estado del hilo del warm-up
El listener empieza a ejecutar el recálculo	RUNNABLE
Dentro del retraso artificial (sleep simulado)	TIMED_WAITING
El retraso termina, sigue calculando	RUNNABLE
El listener termina	TERMINATED

No aparece BLOCKED — ese estado nace de esperar un lock ocupado por otro hilo, y este warm-up no compite por ningún lock.

Problemas de compartición: dos escrituras a la vez

¿Qué pasa si dos escrituras casi simultáneas publican dos eventos? Se lanzan dos tareas de warm-up que pueden ejecutarse a la vez en hilos distintos del pool.

Además del trabajo duplicado, aparece una carrera: si ambas encuentran la caché vacía, una tarea que empezó antes puede terminar después y dejar en Redis un resultado calculado sobre un estado anterior de la base de datos. En un sistema real habría que coordinar esos recálculos.

Problemas de compartición: evento mutable y ThreadLocal

- **¿Y si el listener leyera una entidad JPA compartida en vez de un evento inmutable?**
 - Si el objeto fuera mutable y compartido con el hilo que publicó, existiría el riesgo de que cambiara mientras el listener aún lo usa — la misma condición de carrera de la Actividad 3.1. La inmutabilidad del evento elimina ese riesgo por diseño.
- **¿Y si el listener necesitara el usuario autenticado del hilo original?**
 - No lo tendría — un hilo nuevo no hereda lo que estuviera guardado en un ThreadLocal (así almacena Spring Security el usuario actual). Ese dato tendría que pasarse como parámetro explícito.

El resultado que vas a medir

	Antes (3.2, sin warm-up)	Después (hoy, con warm-up)
Tras crear un libro	La caché queda vacía hasta que alguien vuelve a consultar	El listener empieza a recalentarla automáticamente en segundo plano
¿Quién paga el coste?	El primer usuario que pide el listado tras la escritura	El hilo del warm-up intenta asumir ese coste antes de la siguiente petición

Si el warm-up termina antes de la siguiente petición, ese usuario encontrará ya el resultado en caché. Si una petición llega mientras el warm-up todavía trabaja, puede coincidir con ese recálculo.

Ideas clave

- `@Async` hace que Spring intercepte la llamada y despache la ejecución a un pool de hilos aparte — el llamador no espera a que termine.
- Llamar a un método `@Async` desde dentro de la misma clase hace que Spring no pueda interceptar la llamada: se ignora en silencio y corre en el mismo hilo.
- `@TransactionalEventListener(phase = AFTER_COMMIT)` sincroniza el arranque del listener con el momento en que la transacción ya ha hecho commit.
- El retraso artificial del endpoint lento es un caso observable real del estado `TIMED_WAITING`.
- Publicaciones casi simultáneas pueden provocar trabajo duplicado y carreras entre recálculos; un evento inmutable evita condiciones de carrera sobre los datos compartidos.
- Un hilo nuevo no hereda el contexto del hilo original: nada guardado en un `ThreadLocal` (como el `SecurityContext`) viaja solo — hay que pasarlo como parámetro explícito.